

MERMAID DIAGRAMS

Diagram Reference in Mermaid Syntax

These diagrams can be rendered using Mermaid.js, GitHub markdown, or any Mermaid-compatible tool.

System Architecture

flowchart TB

```
flowchart TB
  subgraph CLI["CLI Layer"]
    main[waft main.py]
    commands[Commands]
  end

  subgraph Core["Core Services"]
    being[Being System]
    corp[Corporation Sim]
    evol[Evolution Engine]
    chron[Chronicler]
    emp[Empirica]
  end

  subgraph Domain["Domain Modules"]
    reality[Reality]
    scint[Scint Detector]
    temple[Templates]
    game[Gamification]
  end

  subgraph Storage["Storage Layer"]
    json[(JSON Files)]
    sqlite[(SQLite DB)]
  end

  CLI --> Core
  Core --> Domain
  Domain --> Storage
```

Being Lifecycle

stateDiagram-v2

```
stateDiagram-v2
    [*] --> SPAWNING: spawn()
    SPAWNING --> LEARNING: initialize
    LEARNING --> EVOLVING: gain_experience
    EVOLVING --> COMPLETING: reach_goals
    EVOLVING --> DEAD: fitness < 0.3
    COMPLETING --> ARCHIVED: archive()
    COMPLETING --> [*]: transcend
    DEAD --> [*]
    ARCHIVED --> [*]

    note right of LEARNING
        Skills increase
        Memories accumulate
    end note

    note right of EVOLVING
        Fitness evaluated
        May mutate
    end note
```

Evolution Cycle

flowchart LR

```
flowchart LR
    A[Spawn Population] --> B[Evaluate in Gym]
    B --> C{Fitness Check}
    C -->|Pass| D[Select Survivors]
    C -->|Fail| E[Death]
    D --> F[Breed Offspring]
    F --> G[Apply Mutations]
    G --> B

    style E fill:#f66
    style D fill:#6f6
```

Scint Detection Flow

flowchart TD

flowchart TD

A[Agent Output] --> B[Pattern Matching]

B --> C{Scint Detected?}

C -->|No| D[Pass Through]

C -->|Yes| E[Classify Type]

E --> F{Type?}

F -->|Syntax| G[SYNTAX_TEAR - 0.3]

F -->|Logic| H[LOGIC_FRACTURE - 0.5]

F -->|Hallucination| I[HALLUCINATION - 0.6]

F -->|Safety| J[SAFETY_VOID - 0.9]

G --> K[Calculate Severity]

H --> K

I --> K

J --> K

K --> L[Update Fitness]

L --> M[Log to Flight Recorder]

Corporation Structure

classDiagram

```
classDiagram
class Corporation {
+corp_id: str
+name: str
+sector: str
+mission: str
+capital: Decimal
+departments: List~Department~
+employees: Dict~str, Employee~
+add_department()
+hire_employee()
+add_revenue()
+add_expense()
}

class Department {
+name: str
+department_id: str
+employees: List~str~
}

class Employee {
+being_id: str
+role: str
+department: str
+title: str
+salary: Decimal
}

class FinancialState {
+capital: Decimal
+transactions: List
+balance_sheet()
+runway_months()
}

Corporation "1" --> "*" Department
Corporation "1" --> "*" Employee
Corporation "1" --> "1" FinancialState
Employee --> Being: references
```

Data Flow

sequenceDiagram

```
sequenceDiagram
    participant User
    participant CLI
    participant Evolution
    participant Gym
    participant FlightRecorder
    participant Storage

    User->>CLI: waft evolve --generations 10
    CLI->>Evolution: start_evolution()

    loop For each generation
        Evolution->>Gym: evaluate(population)
        Gym->>Gym: detect_scints()
        Gym->>Evolution: fitness_scores
        Evolution->>FlightRecorder: log_event(GYM_EVAL)
        Evolution->>Evolution: select_survivors()
        Evolution->>Evolution: breed()
        Evolution->>Evolution: mutate()
        Evolution->>FlightRecorder: log_event(GENERATION)
    end

    Evolution->>Storage: save_checkpoint()
    Evolution->>CLI: results
    CLI->>User: Summary
```

Empirica Session Flow

flowchart LR

```
flowchart LR
    subgraph Preflight
        A[Create Session] --> B[Assess Vectors]
        B --> C[Record Baseline]
    end

    subgraph Work
        D[Execute Task] --> E{Decision Point?}
        E -->|Yes| F[Check Gate]
        F --> G{Proceed?}
        G -->|Yes| D
        G -->|No| H[Revise Approach]
        H --> D
        E -->|No| D
    end

    subgraph Postflight
        I[Reassess Vectors] --> J[Calculate Deltas]
        J --> K[Log Findings]
        K --> L[Update XP/Karma]
    end

    Preflight --> Work
    Work --> Postflight
```

Template System

flowchart TD

```
flowchart TD
    A[Data Source] --> B[Template Registry]
    B --> C{Template Type}

    C -->|Typst| D[Typst Compiler]
    C -->|HTML| E[ Jinja2 Renderer]
    C -->|LaTeX| F[LaTeX Compiler]

    D --> G[PDF Output]
    E --> H[HTML Output]
    F --> G

    subgraph Templates
        T1[tm_report]
        T2[brief]
        T3[memo]
        T4[invoice]
    end

    Templates --> B
```

Entity Relationship

erDiagram

erDiagram

```
REALM ||--o{ CORPORATION : contains
CORPORATION ||--o{ DEPARTMENT : has
CORPORATION ||--o{ EMPLOYEE : employs
EMPLOYEE ||--|| BEING : is_a
BEING ||--o{ MEMORY : has
BEING ||--o{ SKILL : possesses
BEING ||--o{ GOAL : pursues
```

```
REALM {
    string realm_id PK
    string name
    json physics
}
```

```
BEING {
    string being_id PK
    string reality_id FK
    float fitness
    string state
}
```

```
CORPORATION {
    string corp_id PK
    string realm_id FK
    decimal capital
}
```

MERMAID DIAGRAMS | Copy & Paste Ready